Project Saaransh — AI-powered E-Consultation Platform

Smart India Hackathon 2025 • Ministry of Corporate Affairs (MCA) • React • Node/Express • PostgreSQL (Neon) • Python FastAPI + Hugging Face Transformers • deployed on Render

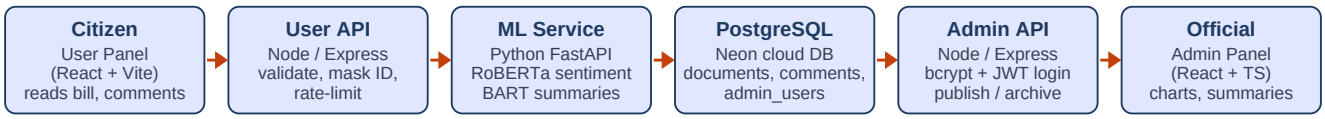
The problem, in plain words

Before the government turns a draft bill into law, it asks the public for comments. Thousands of people reply, and officials have to read every comment by hand to understand what people think. That is slow and easy to get wrong. **Saaransh** (Hindi for “summary”) automates the first reading: it tells officials whether each comment supports, opposes or is neutral on the bill, and writes short summaries of what supporters and critics are saying.

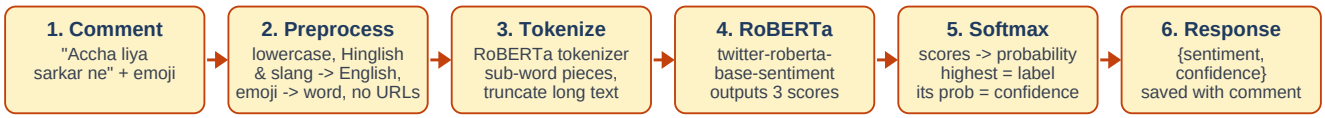
How it works, in plain words

1. Officials publish a bill from the **Admin Panel** (title, dates, text, PDF). It appears on the public site immediately. **2. Citizens** open the **User Panel**, read the bill (with summaries in English, Hindi, Spanish or Tamil, and text-to-speech), and submit a comment with their details. **3.** The backend checks the input, masks the Aadhaar/PAN number, and asks the **Python ML service** to read the comment. **4.** The model returns a sentiment (positive / negative / neutral) and how confident it is; everything is saved in **PostgreSQL**. **5. Officials log in** (email + password) and see charts, trends, stakeholder breakdowns, word clouds and AI summaries, and can archive or delete old bills.

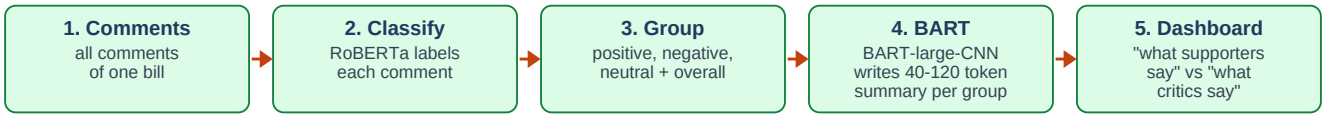
System architecture — citizens on the left, officials on the right, all data in one PostgreSQL database



Python model pipeline — sentiment for every comment (models-new/app.py, FastAPI **POST /predict**)



Summary pipeline — for officials (**POST /summarize-by-sentiment**)



What	Details
Main flows	Publish bill -> citizen comments -> ML sentiment -> admin analytics; archive/delete old bills
Database	3 tables: documents (bills, archived_at), comments (all bills, document_id FK, cascade delete, indexes), admin_users
Security	bcrypt + JWT login, requireAuth on admin APIs, input validation, rate limits, Helmet, masked Aadhaar/PAN, parameterised SQL
My contributions	*[Fill in: e.g. backend restructure, auth, dynamic bills, security, ML integration]*

60-second pitch (say this when asked "Tell me about your project")

"Saaransh is an AI-powered e-consultation platform we built for the Ministry of Corporate Affairs at Smart India Hackathon 2025. When the government publishes a draft law, thousands of citizens comment, and officials read them manually. We built a public portal where citizens read bills in four languages with text-to-speech and submit feedback, and an admin dashboard where officials publish bills and see that feedback analysed. Every comment goes through a RoBERTa transformer for sentiment, and a BART model summarises what supporters and critics say. The stack is React, Node/Express, PostgreSQL on Neon and a Python FastAPI ML service on Render. I worked on [your part], and I also hardened it: bcrypt/JWT admin login, input validation and rate limiting, masked Aadhaar storage, and a single comments table so new bills need no code changes."

Interview Questions & Answers

Answer in 30-60 seconds, then stop. Follow-ups are what interviewers usually ask next.

The project and your role

Q. Tell me about this project.

Saaransh is an e-consultation platform for the Ministry of Corporate Affairs. Officials publish draft bills; citizens read them and submit comments; a transformer model classifies each comment's sentiment and a summarisation model condenses supporters' and critics' views, so officials understand thousands of comments in minutes. It has two React apps, two Express backends, PostgreSQL on Neon, and a Python FastAPI ML service, deployed on Render.

Follow-up: *What exactly did you build?* — Name 2-3 concrete pieces and one decision for each (e.g. 'I designed the single comments table and the migration', 'I replaced the demo login with bcrypt + JWT').

Q. Who are the users and what does each need?

Citizens need a simple, fast, multilingual site to read a bill and comment, without creating an account. Officials need secure login, analytics, summaries and control over which bills are open. That difference is why there are two separate apps and backends.

Q. What was the hardest part?

Making the system work for any number of bills. Originally there was one table and one hand-built page per bill, so a new bill needed code changes. I moved all comments into one table keyed by document_id, migrated the existing 49 comments safely (copy, verify counts, deploy, re-copy, then drop), and made both frontends load bills from the API.

Follow-up: *How did you avoid losing data during the migration?* — Full backup first; the copy script is idempotent (legacy_ref UNIQUE + ON CONFLICT DO NOTHING); it verifies row counts before dropping anything; I re-ran it after deploy to catch comments submitted in between.

Architecture and design choices

Q. Walk me through the architecture.

Browser -> React frontend -> Express REST API -> PostgreSQL. The user backend also calls the FastAPI ML service over HTTP for sentiment. The admin app has its own backend protected by JWT. Both backends share one Neon database. Frontends find their backend through the VITE_API_URL environment variable.

Follow-up: *Why two backends instead of one?* — Separation of concerns and security: the public API returns no personal data and has no admin routes; the admin API requires login on every data route.

Follow-up: *Is it microservices?* — Service-oriented: three services with clear jobs talking over HTTP. True microservices would also have separate databases.

Q. Why this tech stack?

React for component-based UI; Node/Express because the backends are I/O-bound (DB + ML calls) and share a language with the frontend; PostgreSQL because the data is relational (bills have many comments) and we need aggregations like GROUP BY sentiment; Python FastAPI because the ML ecosystem (PyTorch, Transformers) is Python.

Follow-up: *Why not MongoDB?* — Fixed schema, foreign keys, cascade deletes and SQL aggregations fit this data better.

Follow-up: *Why not run the model in Node?* — Models and tooling are Python-first, and a separate service can be scaled or moved to a GPU independently.

Q. How is the backend code organised?

Layered: routes (URL + middleware), controllers (request logic), services (ML, OTP, summaries), models (SQL), middleware (auth, validation, rate limits, errors), utils (masking, normalising). server.js/index.js only load config and start src/app.js, so the app can be tested without opening a port.

Q. How do frontend and backend talk?

HTTP with JSON. All calls go through one service file (services/api.js, lib/api.ts). On the admin side that file adds the 'Authorization: Bearer token' header and logs the user out on a 401.

Follow-up: *What is CORS?* — A browser rule blocking a page from reading another origin's response unless the server allows it. The public API allows any origin because it uses no cookies; protection comes from validation and rate limits. The admin API is protected by JWT, not by CORS.

The feedback flow

Q. What happens when a citizen clicks Submit?

The modal validates the form in the browser, then POSTs to /api/submit-comment. Express runs Helmet, CORS, the rate limiter, the JSON parser and express-validator rules. The controller checks the bill exists and is not archived, calls the ML service, masks the ID number, normalises section and stakeholder labels, and runs a parameterised INSERT into comments. It returns only the new comment id and the sentiment.

Follow-up: *Why validate twice?* — Browser validation is for user experience; server validation is for security, because anyone can call the API directly.

Follow-up: *What if the ML service is down?* — The call has a timeout; the comment is saved as neutral so the citizen never loses it. Better: save as 'pending' and re-process from a queue.

Q. You built OTP verification. Why is it off?

It uses Twilio Verify (6-digit code, 10-minute expiry, 60 s resend cooldown, 5 sends/hour per number, single use). Real SMS costs money and needs Twilio setup, so it sits behind an OTP_ENABLED flag: turning it on is configuration, not code.

Follow-up: *Why hash the OTP and use timingSafeEqual?* — A memory dump won't reveal codes, and constant-time comparison prevents timing attacks.

The machine learning model

Q. Explain the sentiment model.

We use cardiffnlp/twitter-roberta-base-sentiment, a RoBERTa transformer fine-tuned on tweets for 3 classes. Text is preprocessed, tokenised into sub-words, passed through the model to get 3 logits; softmax turns them into probabilities; the highest is the label and its probability is the confidence.

Follow-up: *Why a tweet model?* — Public comments are short, informal and opinionated, like tweets.

Follow-up: *Why not VADER or Naive Bayes?* — Transformers understand context and negation ('not bad'); bag-of-words and lexicon methods miss that. Trade-off: more compute.

Q. How do you handle Hinglish and emojis?

The model is English-only, so preprocessing maps common Hinglish words and phrases to English ('accha' -> good, 'virodh' -> oppose), expands slang, converts emojis to words (thumbs-up -> 'support') and strips URLs.

Follow-up: *Limits?* — Only words in the dictionary are handled. Better: a multilingual model such as MuRIL or XLM-RoBERTa, or fine-tuning on labelled Indian comments.

Q. How does summarisation work?

Comments of a bill are classified, grouped into positive, negative and neutral, and each group is summarised by BART-large-CNN (abstractive, 40-120 tokens). Officials see what supporters and critics say separately.

Follow-up: *BART's input limit?* — About 1024 tokens, so input is cut to 3000 characters. For thousands of comments I'd use map-reduce: summarise chunks, then summarise the summaries.

Follow-up: *Abstractive vs extractive?* — Abstractive writes new sentences (more natural, can hallucinate); extractive copies sentences. Officials can always open the original comments.

Q. How did you measure accuracy?

There is a /heatmap endpoint that builds a confusion matrix from labelled samples. Only quote numbers you actually measured; the honest next step is evaluating on a larger labelled set of real consultation comments.

Follow-up: *What bug did you find around confidence?* — Every comment showed 4.2/5: a hard-coded placeholder column, and /predict didn't return a confidence at all. I made /predict return the softmax probability and the dashboard convert it to a 0-5 scale.

Q. How would you make inference faster or cheaper?

Batch comments, run on GPU, use a distilled model (DistilRoBERTa), quantise or export to ONNX, and process asynchronously through a queue so users never wait for the model.

Database

Q. Describe the schema.

documents (one row per bill, archived_at for soft-hiding), comments (every comment of every bill: text, sentiment, confidence, keywords JSONB, masked ID, stakeholder type, document_id foreign key with ON DELETE CASCADE), and admin_users (bcrypt hashes). Indexes on (document_id, created_at) and (document_id, sentiment).

Follow-up: *Why did you replace one-table-per-bill?* — Every new bill needed a new table and code changes, and cross-bill queries needed UNIONs. One table with a foreign key is normalised and scales to any number of bills.

Follow-up: *What is an index and its trade-off?* — A B-tree that turns full scans into ~O(log n) lookups; costs storage and slower writes.

Q. Archive vs delete - why both?

Archive (soft delete) hides a bill from citizens but keeps comments for audit and later analysis - important for government records. Delete is optional, confirmed in the UI, and removes the bill and its comments via cascade.

Follow-up: *Will old data slow the database?* — Not at this scale: PostgreSQL handles hundreds of thousands of rows easily with the document_id index; deletion is a data-policy choice, not a performance need.

Q. Name a subtle bug you fixed.

Dates showed one day early. The pg driver converted DATE columns to JavaScript Dates at local midnight, and toISOString shifted them to the previous day in IST. I made the driver return DATE as a plain 'YYYY-MM-DD' string.

Authentication and security

Q. How does admin login work?

Sign-up stores the profile and a bcrypt hash in admin_users. Login compares the password with bcrypt and returns a JWT (HS256, 8 h) signed with JWT_SECRET. The frontend sends it as a Bearer token; requireAuth middleware verifies it on every data route. An expired token returns 401 and the frontend logs out.

Follow-up: *Why bcrypt?* — Slow, salted hashing - brute-forcing a leaked hash is expensive.

Follow-up: *Where is the JWT secret kept?* — In an environment variable (Render settings / local .env), never in the public repo, and stable across restarts so sessions survive deploys.

Follow-up: *JWT vs sessions?* — JWT is stateless - no session store - but cannot be revoked early; short expiry limits risk. Storing it in an httpOnly cookie would protect better against XSS than localStorage.

Q. What security measures did you add?

Strict validation (phone, email, Aadhaar/PAN formats, allowed values, lengths), per-IP rate limits (login 10 per 15 min, sign-up 5 per hour, submissions 20 per 15 min), Helmet headers, parameterised SQL, masked Aadhaar/PAN, generic login errors with a dummy bcrypt compare (no user enumeration), safe 5xx messages, SSRF allow-list on the audio proxy, and server-side sentiment only.

Follow-up: *What security problems existed before?* — Public endpoints returned every citizen's Aadhaar and phone, the admin login was a hard-coded demo account, admin APIs had no auth, and an old table stored plain-text passwords. I fixed or removed all of them.

Follow-up: *What is SSRF?* — Tricking a server into fetching an attacker's URL; the TTS proxy only fetches HTTPS Google TTS hosts.

Frontend (React)

Q. How is the React app structured?

Pages (listing, filtered list, one generic DocumentPage for any bill), layout components, a CommentModal with a multi-step form, shadcn/ui components, a useConsultations hook that loads bills once, and one API file. React Router maps URLs like /documents/:id to pages.

Follow-up: *Why a custom hook?* — The listing, filters and landing counts all need the same bill list; the hook keeps fetching, loading and error state in one place.

Follow-up: *How do you protect admin pages?* — A ProtectedRoute checks the auth context; on load the stored token is verified with GET /api/auth/me before the dashboard renders.

Q. What is useEffect and when does it run?

It runs side effects (like API calls) after render. With an empty dependency array it runs once on mount; with dependencies it re-runs when they change. I return a cleanup function to ignore results after unmount.

Q. How is the API URL configured per environment?

Vite injects VITE_API_URL at build time, so the same code calls localhost in development and the Render URL in production. Changing it requires a rebuild.

Node.js and REST APIs

Q. Node is single-threaded - how does it serve many users?

The event loop hands I/O (database, HTTP to the ML service) to the OS / libuv and runs callbacks when they finish, so one thread serves many concurrent requests. CPU-heavy work would block it - one reason the model runs in a separate Python service.

Q. What is Express middleware?

A function (req, res, next) in a pipeline that can read or change the request, end the response, or call next(). Order matters: Helmet and CORS first, then rate limiting, body parsing, validation, the route, and finally the 4-argument error handler.

Q. Which HTTP methods and status codes do you use?

GET to read, POST to create (201), PATCH to archive (partial update), DELETE to remove. 400 for invalid input, 401 when not logged in, 404 when a bill does not exist or is archived, 409 for a duplicate email, 429 when rate limited, 500 for unexpected errors (generic message only).

SQL

Q. Write a query: sentiment counts for one bill.

`SELECT sentiment, COUNT(*) FROM comments WHERE document_id = $1 GROUP BY sentiment;` The `$1` placeholder is a parameter, which prevents SQL injection, and the (document_id, sentiment) index makes it fast.

Q. How do you count comments per bill including bills with zero comments?

`SELECT d.document_id, COUNT(c.comments_id) FROM documents d LEFT JOIN comments c ON c.document_id = d.document_id GROUP BY d.document_id;` A LEFT JOIN keeps bills without comments; COUNT(column) ignores NULLs, so they show 0.

Follow-up: *What does ON DELETE CASCADE do?* — Deleting a bill automatically deletes its comments, so no orphan rows remain.

Q. What is normalisation and did you apply it?

Storing each fact once and linking with keys. Moving from three copies of the same table to one comments table with a document_id foreign key is exactly that; I also normalised values (one spelling of each stakeholder type) so GROUP BY results are not split.

More on the ML model

Q. Which metrics would you use to evaluate the classifier?

Accuracy, plus precision, recall and F1 per class, and the confusion matrix - accuracy alone hides problems when classes are imbalanced (e.g. many neutral comments).

Follow-up: *Precision vs recall?* — Precision: of comments predicted negative, how many really are. Recall: of all truly negative comments, how many we caught. F1 is their harmonic mean.

Q. What is fine-tuning and would it help here?

Continuing to train a pretrained model on your own labelled data. Yes: policy comments use legal vocabulary that tweets don't, so fine-tuning on a few thousand labelled consultation comments should improve accuracy.

Follow-up: *Risk?* — Overfitting on a small dataset - use a validation split, early stopping and a low learning rate.

Q. What is a tokenizer and why sub-words?

It splits text into pieces the model knows. Sub-word (BPE) tokens let the model handle rare or new words by combining known pieces instead of marking them unknown. RoBERTa accepts up to 512 tokens, so long comments are truncated.

Testing and tooling

Q. How did you test your changes?

API tests with curl against a local server: valid and invalid input, wrong OTP, expired token, archived bill, and an end-to-end flow (publish, comment, archive, restore, delete) that cleans up after itself. Database writes during tests were stubbed or deleted afterwards. Frontends were verified with production builds, TypeScript checks and ESLint.

Follow-up: *What would you automate?* — Jest + supertest API tests in CI on every push, and pytest for the model service.

Q. How do you use Git?

Small focused commits with clear messages, secrets kept out via .gitignore (.env), and every deploy traceable to a commit because Render builds from main.

Deployment and debugging

Q. How is it deployed?

Render auto-deploys each service from GitHub main. Secrets (DATABASE_URL, JWT_SECRET) are environment variables. The admin backend creates missing tables on start (CREATE TABLE IF NOT EXISTS), so a deploy never needs a manual DB step.

Q. Tell me about a production bug.

Comments worked locally but failed on the live site. I probed the live API and found the backend rejected the site's origin: a stale CORS_ORIGIN value that my new code had started enforcing. Since the API is public and cookie-less, I allowed all origins and kept validation and rate limits as the real protection, then verified the deployed fix.

Follow-up: *What did you learn?* — Test the production configuration, not just local, and roll out stricter checks with a safe default.

Scaling and improvements

Q. How would you handle 1 million comments in a week?

Stateless backends behind a load balancer; save comments instantly and push ML work to a queue (SQS) with GPU workers processing batches; Redis for rate limits; read replicas and pre-computed counts for the dashboard; CDN for frontends and PDFs.

Follow-up: *Bottleneck today?* — The ML call - inference is far slower than the INSERT and the request waits for it.

Follow-up: *How do you not lose work if a worker crashes?* — Queue acknowledgements and visibility timeouts; repeated failures go to a dead-letter queue.

Q. What would you improve next?

Automated tests (Jest + supertest, pytest), async ML via a queue, map-reduce summaries, a multilingual or fine-tuned model, httpOnly-cookie sessions, role-based admin access and restricted sign-up (ALLOWED_SIGNUP_DOMAINS), and auto-closing comments after the due date.

Q. How would you deploy on AWS?

S3 + CloudFront for frontends, ECS/Fargate behind an ALB for the APIs, RDS PostgreSQL, a SageMaker or GPU endpoint for the model, SQS for jobs, ElastiCache Redis, Secrets Manager, CloudWatch for logs and alarms.

Behavioural (Leadership Principles) from this project

Q. Ownership - fixing something not assigned to you.

I noticed public endpoints exposing citizens' Aadhaar numbers. I verified nothing used them, removed them, masked IDs at storage, and added validation and rate limits - without breaking any feature.

Q. Dive Deep - a bug you investigated.

The 'every comment is 4.2/5 confidence' issue: I traced it from the dashboard to a placeholder column and to the ML endpoint not returning confidence, fixed both ends, and documented it.

Q. Customer Obsession.

Summaries in four languages with text-to-speech and Hinglish-aware preprocessing, so citizens who are not comfortable with English legal text can still participate and be understood correctly.

Q. Bias for Action / Deliver Results.

In the hackathon we used pretrained models and a component library to ship end-to-end on time, then later paid down the shortcuts (one table per bill, demo login) with careful, verified migrations.

Tip: use STAR (Situation, Task, Action, Result), say "I" for your own work, and never invent numbers.